



ADVANCED DATABASE SYSTEMS

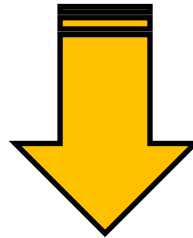
WEEK 10

QUERY OPTIMIZATION PART IV



Query Optimization

For a given SQL Query



Goal:

Build the most “optimal” execution plan for the query



Learning to Optimize Query Processing

Learning to optimize query processing

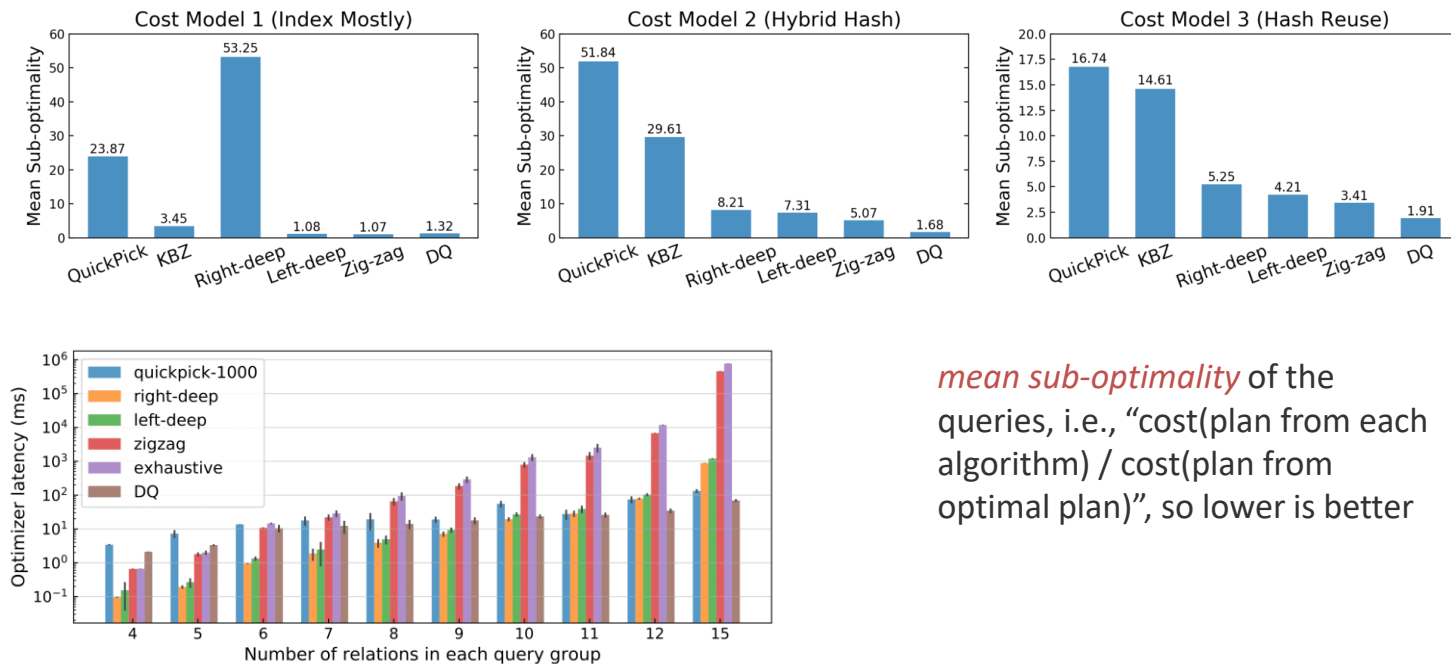
01 Is database a solved problem?

DBMS is a solved problem

- > 50 years of DB research
- We developed
 - fantastic algorithms
 - great systems
 - clever query processing and storage strategies
 - a lot of brilliant stuff
- DBMSs have become fast
 - when you look at TPC-C: we are currently able to execute ~half a million transactions per second
 - to simple index-lookups, e.g., in a hash table, we are currently at 20 million operations per second

DBMS is not solved yet

Take query optimization as an example, DQ outperform **1.32X** of traditional methods



mean sub-optimality of the queries, i.e., “cost(plan from each algorithm) / cost(plan from optimal plan)”, so lower is better

- Sanjay Krishnan et al. [Learning to Optimize Join Queries With Deep Reinforcement Learning](#). arXiv preprint, 2019.

02 Query optimization with RL

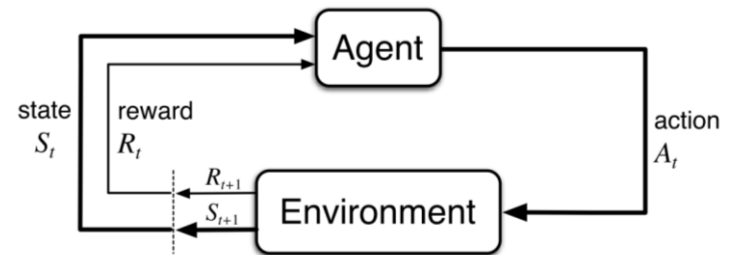
Automatic query optimization

- Traditional query optimization
 - Relational algebraic equivalence
 - Join order enumeration
- Adaptive query processing
 - Traditional optimize-then-execute paradigm
 - Data is changing over time
- Learning to optimize with deep reinforcement learning
 - DQ (Deep Q-learning)
 - Neo (Neural Optimizer)
 - SkinnerDB

Reinforcement learning

- RL Set-Up

- Agents interacts with the environment by *taking actions* and *receiving feedbacks*
- Feedback is in the form of *rewards*
- Agent's utility is defined by the *reward function*
- Must (learn to) act to *maximize expected rewards*



Reinforcement learning

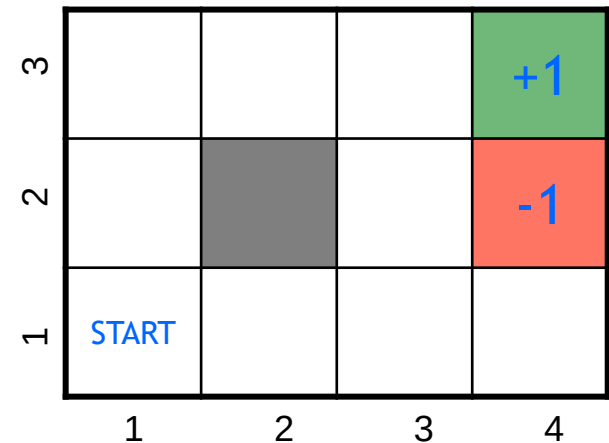
- transitions and rewards usually not available
- how to change the policy based on experience
- how to explore the environment

Markov Decision Process (MDP)

- Set of **states** S , set of **actions** A , initial **state** S_0
- Transition model $P(s,a,s')$
 - $P([1,1], \text{up}, [1,2]) = 0.8$
- Reward function $r(s)$
 - $r([4,3]) = +1$
- Goal: maximize cumulative reward in the long run
- Policy: mapping from S to A
 - $\pi(s)$ or $\pi(s,a)$ (deterministic vs. stochastic)

MDP solvers

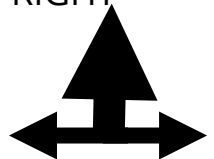
- Dynamic programming
- Monte Carlo methods
- Deep Q-learning



- reward +1 at [4,3], -1 at [4,2]
- reward -0.04 for each step
- what's the strategy to achieve max reward?

actions: UP, DOWN, LEFT, RIGHT

80% move UP
10% move LEFT
10% move RIGHT



DQ: optimize join with deep Q-learning

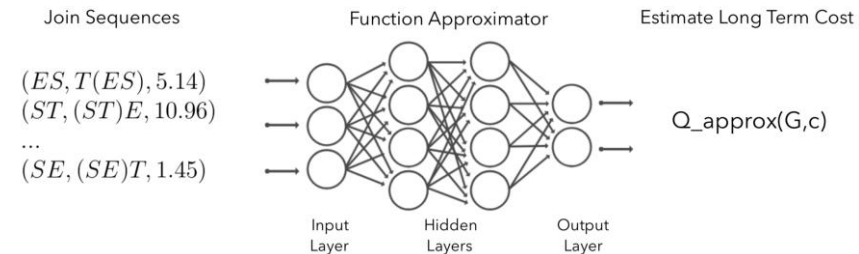
RL can be modeled as a Markov Decision Process (MDP)

Formulate join ordering as an MDP (G, c, G', J)

- **States G** : the remaining relations to be joined.
- **Actions c** : a valid join out of the remaining relations.
- **Next states G'** : naturally, this is the old “remaining relations” set with two relations removed and their resultant join added.
- **Reward J** : estimated cost of the new join.

Apply Q-learning to solve the join-ordering MDP

- Q-function: $Q(G, c) = J(c) + \min_{c'} Q(G', c')$
- describes the long-term cost of each join: the **cumulative cost** if we act optimally for all subsequent joins after the current join decision.



Learning algorithm:

- (1) Start with the initial query graph,
- (2) Find the join with the lowest $Q(G, c)$,
- (3) Update the query graph and repeat.

Training the optimizer in DQ

To learn the Q-function (Training the model)

1. **Training data**: To learn the Q-function we first need to observe past execution data. DQ can accept a list of (G, c, G', J) from any underlying optimizer (**run a classical left-deep dynamic program**).

- For example: $(G, c, G', J) = (\{E, S, T\}, \text{join}(S, T), \{E, ST\}, 110)$ means that we start with the initial query graph (**state**) and joining S and T together (**action**).

2. Featurization of **states** and **actions**

- Feed states G and actions c into the network as *fixed-length feature vectors*
- 1-hot vectors** to encode: (1) the set of all attributes A_G present in the query graph, out of all attributes in the schema, (2) the participating attributes A_L from left side of the join, and (3) those A_R from right side of the join

```
SELECT *
FROM Emp, Pos, Sal
WHERE Emp.rank =
      Pos.rank
AND Pos.code = Sal.code
```

(a) Example query

$$A_G = [E.id, E.name, E.rank, \\ P.rank, P.title, P.code, \\ S.code, S.amount] \\ = [1\ 1\ 1\ 1\ 1\ 1\ 1\ 1]$$

(b) Query graph featurization

$$A_L = [E.id, E.name, E.rank] \\ = [1\ 1\ 1\ 0\ 0\ 0\ 0\ 0] \\ A_R = [P.rank, P.title, P.code] \\ = [0\ 0\ 0\ 1\ 1\ 1\ 0\ 0]$$

(c) Features of $E \bowtie P$

$$A_L = [E.id, E.name, E.rank, \\ P.rank, P.title, P.code] \\ = [1\ 1\ 1\ 1\ 1\ 1\ 0\ 0] \\ A_R = [S.code, S.amount] \\ = [0\ 0\ 0\ 0\ 0\ 0\ 1\ 1]$$

(d) Features of $(E \bowtie P) \bowtie S$

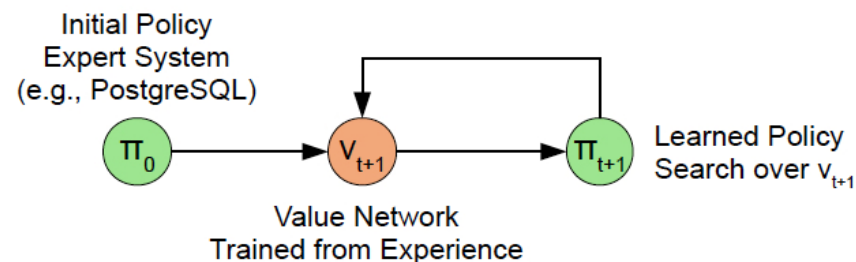
Neo: a learned query optimizer

DQ

- Only optimize the join ordering
- DQ uses a fully connected NN to approximate the Q-function
- The model is trained with a standard stochastic gradient descent (SGD) algorithm
- DQ train the optimizer from scratch

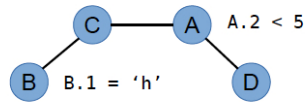
Neo

- Neo optimize join ordering and execution plan
- Neo use a tree CNN instead of fully connected NN used by DQ
- The model was trained via value iteration
- Neo bootstraps its query optimization model from existing optimizers (PostgreSQL's)



Neo (cont.)

SELECT * FROM A, B, C, D WHERE
A.3=C.3 AND A.4=D.4 AND C.5=B.5
AND A.2<5 AND B.1='h';



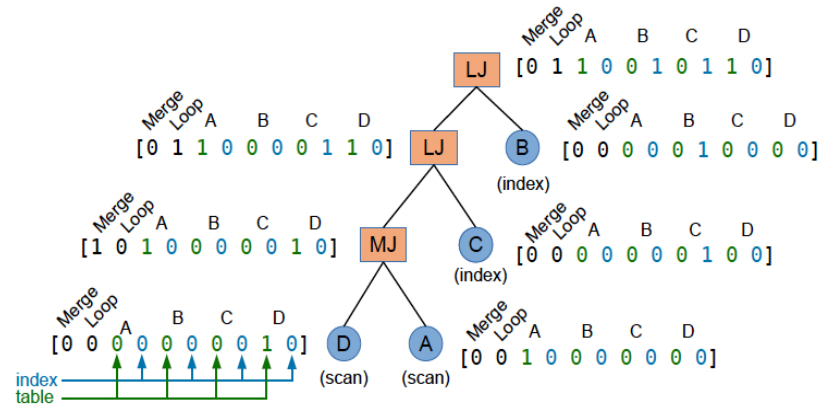
	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	0	0
C	1	1	0	0	0
D	1	0	0	0	0
E	0	0	0	0	0

A.1	A.2	...	B.1	B.2	...	E.1	E.2
0	1	...	1	0	...	0	0

Column Predicates

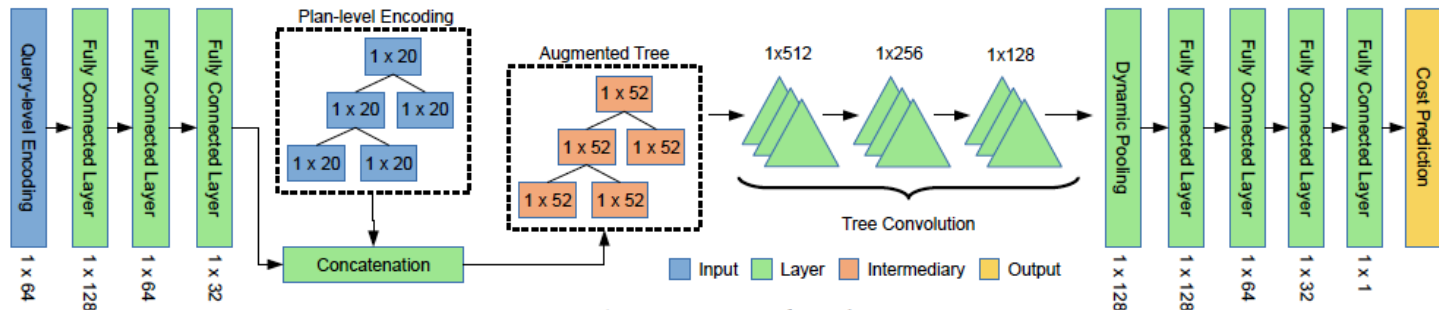
[0 1 1 0 1 0 0 0 0 0 0 1 ... 1 0 ... 0 0]

Query-level encoding



Plan-level encoding

Value network (tree CNN)



SkinnerDB: learning by doing

ML for query optimization:

Inter-query learning: DQ, Neo

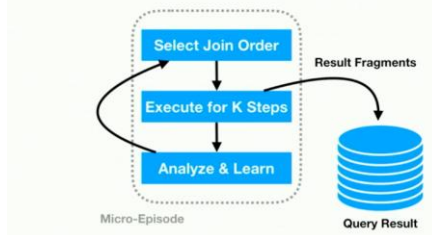
- 🔗 Train the query optimizer with past queries

Intra-query learning: SkinnerDB

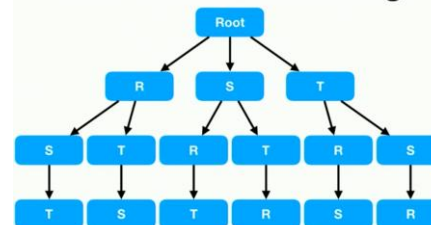
- 🔗 Learn optimal join orders on the fly, during the execution of the current query
- 🔗 Divide the execution of a query into many small time slices
- 🔗 Different join orders are tried in different time slices (UCT search tree)
- 🔗 Merge result tuples generated according to different join orders until a complete result is obtained
- 🔗 SkinnerDB can converge to optimal join orders with regret bounds

	<i>Inter-query</i> learning	<i>Intra-query</i> learning
Training	Past queries	Current query
Application	Current query	Current query

Intra-Query Learning



UCT for Join Ordering



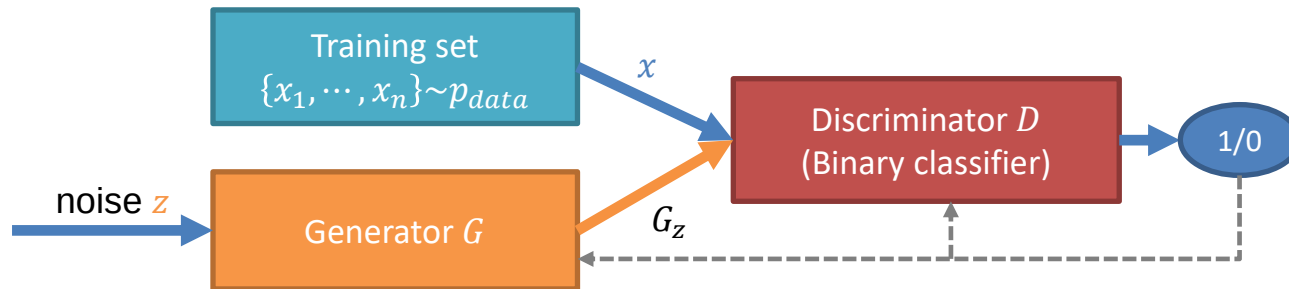
04 Adversarial training for query optimizer

1. Generative adversarial networks
2. Monte Carlo tree search.
3. AlphaGo : D. Silver, et al. Mastering the game of go with deep neural networks and tree search. In Nature. Nature Research, 2016.

Shortcomings of the existing methods

- **Shortcoming 1:** lack of training data
 - DQ learns from scratch and takes very long time for training
 - Neo relies on the existing knowledge but with limited improvement
 - SkinnerDB tries to learn an adaptive execution by dividing the data into many small slots and each slot acts a distinct execution plan
- **Shortcoming 2:** Lack of capability to tradeoff exploitation and exploration
 - DQ relies on deep Q-learning tries to exhaust all enumerations which is impossible for large queries
 - Neo relies on value iteration which takes a long time to converge
- **Another solution**
 - Generative adversarial neural network (GAN) for the shortcoming 1
 - Monte Carlo search tree (MCTS) to address the shortcoming 2

Generative adversarial network (GAN)

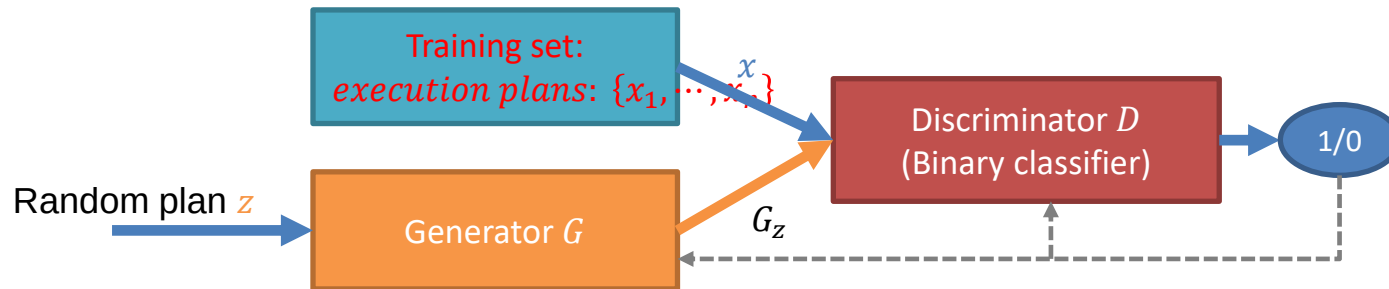


MinMax Game (Zero-sum gaming)

- Generator tries to fool discriminator (i.e. generate realistic samples)
- Discriminator tries to distinguish fake from real samples
- Each tries to minimize the objective function maximized by the other

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Query optimization via adversarial training



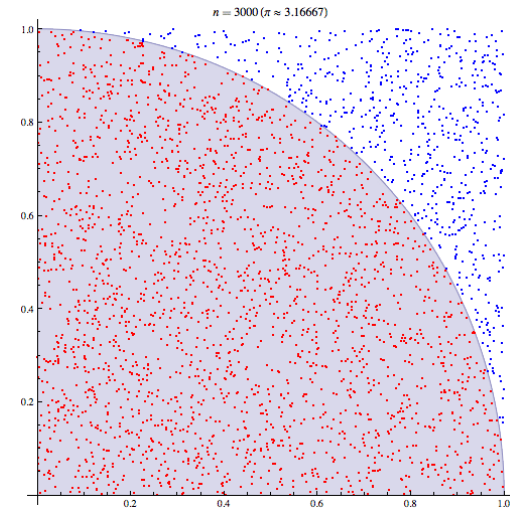
MinMax Game

- The discriminator D is a cascaded joins of value network connecting all tables together
- The generator G is a cascaded series of the MCTS improved policy (i.e. generate realistic samples)
- Each tries to minimize the objective function maximized by the other

$$\min_G \max_D (D, G) = \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

Monte Carlo Tree Search (MCTS)

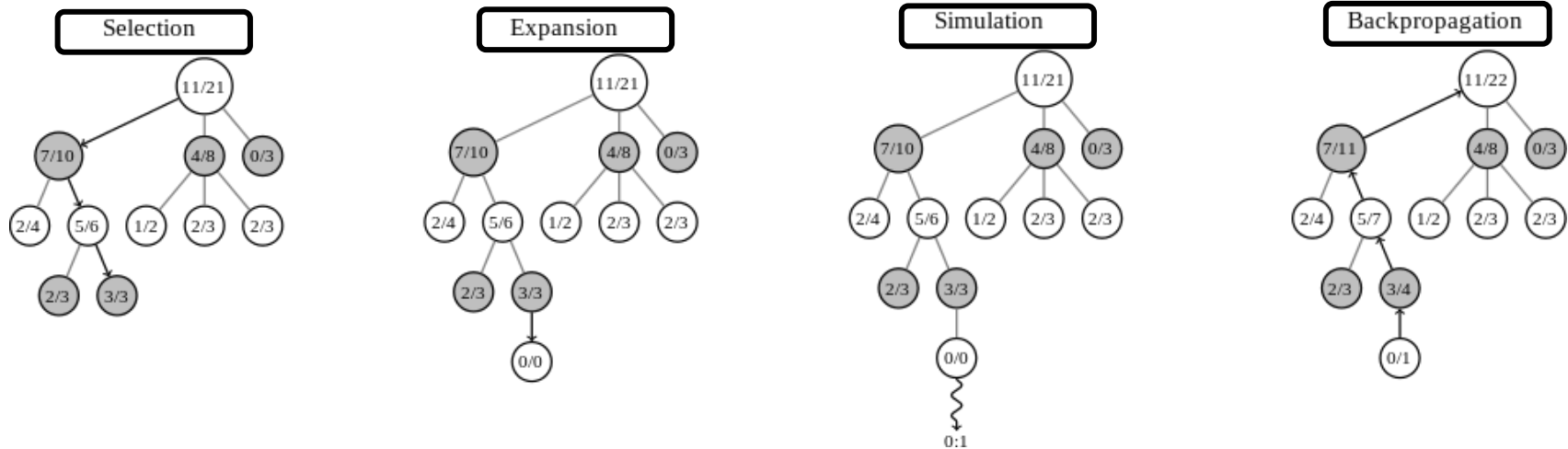
- Monte Carlo Experiments : repeated random sampling to obtain numerical results
- Search method
- Method for making optimal decisions in artificial intelligence (AI) problems
- The strongest Go AIs (Fuego, Pachi, Zen, and Crazy Stone) all rely on MCTS



Monte Carlo Tree Search

Each round of MCTS consists of four steps

1. **Selection**: start from root R and select successive child nodes until a leaf node L is reached
2. **Expansion**: create one (or more) child nodes and choose node C from one of them
3. **Simulation**: play a random rollout (choosing a uniform random move) from node C
4. **Backpropagation**: update information in the nodes on the path from C to R



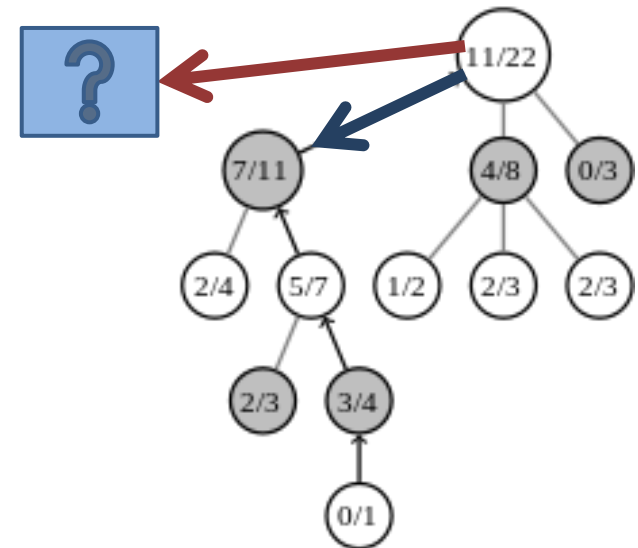
MCTS – Upper Confidence Bounds for Trees

- For every child we calculate the following function to tradeoff exploration and exploitation
- Convergence to the optimal solution
 - Kocsis, L. & Szepesvári, C. Bandit based Monte-Carlo planning (2006)

Exploitation **Exploration**

$$\frac{\omega_i}{n_i} + c \cdot \sqrt{\frac{\ln(t)}{n_i}}$$

W_i #wins after visiting the node i
 n_i #times node i has been visited
 C exploration parameter
 t #times node i parent has been visited



Experimental studies

x	o	x
o	x	o
x	o	x

A tik-tac-toe game with 10 columns and 10 rows

- The training stops when agent win over 80%

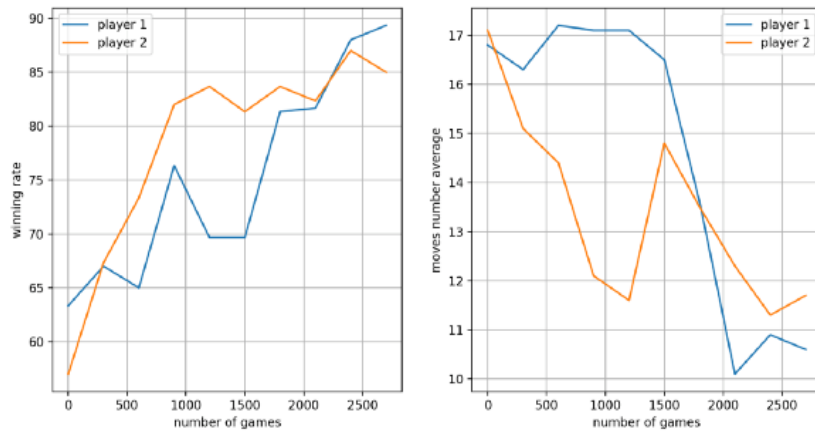
Learning process of Deep Q-Learning:

1. Offline learning with planning, where batches of state-action pairs were stored and the network was trained on these batches
2. The playing policy was an ϵ -greedy policy which means that an exploration factor ϵ was chosen
3. Take ReLu function as an activation function
4. The agent played 300 episodes against random player and measured the winning rate

Learning process of MCTS:

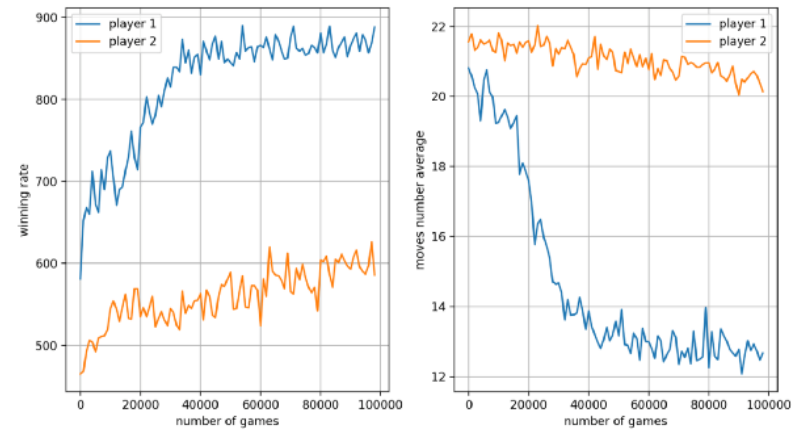
1. The agent takes 1M episodes, but the learning is a lot faster (a few hours)
2. To store a tree of 1M episodes it takes around 800MB and it is growing when the agent continues learning. So it is a lot more than the CNN which stayed constant through all the learning process.
3. The graph below shows the winning rate of the MCTS agent against random player as the learning progresses.

Experimental studies



Deep Q-learning

Player 1: DQN agent
Player 2: random agent



MCTS

Player 1: MCTS agent
Player 2: random agent